studocu

SE601 Exam Preparation Summary from slides through gpt

Software Development (Virtual University of Pakistan)



Scan to open on Studocu

Studocu is not sponsored or endorsed by any college or university
Downloaded by sana umeed (sanaumeed86@gmail.com)

SE601 Exam Preparation Summary
Created by Umar
Dated : 2025

It's the over view of key concepts and important topic from week 7 to week 16 for final term preparation

Week 7: Software Construction and Development

Key Topics:

1.

Efficiency of Use and Performance

2.

1. A system should minimize steps required to complete tasks.
2. Example: A smartphone dialer should allow calling a new contact quickly without unnecessary steps.

3.

Aesthetic and Minimalist Design

4.

1. UI should not contain unnecessary elements.
2. Example: Auto-moving the cursor to the next input field in a form.

5.

Flexibility and Efficiency of Use

6.

1. System should provide shortcuts for expert users.
2. Example: Swipe gestures or double-tap to zoom in maps.

7.

Handling Varied Contexts in Mobile Environments

8.

1. Interface should be adaptable to different conditions (light, noise, motion).

Downloaded by sana umeed (sanaumeed86@gmail.com)

2. Example: Increasing ringtone volume when in motion.

9.

Recognition vs Recall

10.

1. Users should recognize actions instead of memorizing steps.
2. Example: A visible delete button instead of requiring a keyboard shortcut.

Possible Exam Questions:

1.

What is the benefit of flexibility in software design?
Answer: Flexibility allows users to customize and speed up interactions using shortcuts, gestures, and automation.

2.
3.

Explain the importance of aesthetic and minimalist design.
Answer: It enhances usability by reducing distractions and making key elements easy to find.

4.

Week 8: Object-Oriented Design Principles

Key Topics:

1.

Encapsulation & Information Hiding

2.

1. Protects data by restricting direct access.
2. Example: A bank account balance is private, and only accessible via `getBalance()`.

3.

Downloaded by sana umeed (sanaumeed86@gmail.com)

Abstraction

4.

1. Hides unnecessary details and exposes only relevant information.
2. Example: A `Car` class exposes `drive()`, but hides engine mechanics.

5.

Inheritance & Polymorphism

6.

1. **Inheritance:** Child classes inherit features from parent classes.
2. **Polymorphism:** Different objects respond to the same method differently.
3. Example: A `Shape` class has a `draw()` method, which is implemented differently in `Circle` and `Rectangle`.

7.

SOLID Principles

8.

1. **Single Responsibility Principle:** A class should have only one reason to change.
2. **Open/Closed Principle:** Classes should be open for extension but closed for modification.

Possible Exam Questions:

1.

What is the difference between inheritance and polymorphism?
Answer: Inheritance allows one class to acquire properties of another, while polymorphism enables different objects to implement the same method differently.

2.
3.

Explain the Single Responsibility Principle with an example.
Answer: A `User` class should handle user data, while a separate `UserDatabase` class should manage database operations.

Downloaded by sana umeed (sanaumeed86@gmail.com)

Week 9 (Part 1): Coding Standards

Key Concepts

1.

What Are Coding Standards?

2.

1. A set of guidelines for writing **clean, readable, and maintainable** code.
2. Covers **naming conventions, indentation, comments, file structure**, etc.
3.

Why Use Coding Standards?

4.

1. Improves **readability** and makes code easier to maintain.
2. Reduces **bugs and errors** in software.
3. Helps with **team collaboration** and reduces maintenance costs.

5.

Naming Conventions

6.

1. **Variables:** Use meaningful names (e.g., `numStudents`, not `x`).
2. **Functions:** Start with an action word (e.g., `getName()` instead of `name()`).
3. **Classes:** Begin with an **uppercase letter** (e.g., `CustomerAccount`).

7.

Indentation & Spacing

8.

1. Use **4 spaces or a tab** for indentation.
2. Use **spaces around operators** (`x = y + 8;` instead of `x=y+8;`).

9.

Comments & Documentation

10.

1. **Every .cpp and .h file must have a file header comment** (file name, date, author, description).

Downloaded by sana umeed (sanaumeed86@gmail.com)

2. **Inline comments** should appear above the code they refer to.
3. **Every function should have a function header comment**, except getters/setters.

Possible Exam Questions

1.

What are coding standards, and why are they important?
Answer: Coding standards define best practices for writing code to improve readability, reduce bugs, and simplify maintenance.

2.
3.

What is the recommended indentation style in coding standards?
Answer: Use **4 spaces or a tab** per indentation level for consistency.

4.

Week 9 (Part 2): Professional Ethics

Key Concepts

1.

What is Professional Ethics in Software Engineering?

2.

1. Ethical responsibilities include **privacy, security, reliability, and safety**.
2. Ensures that software does not cause **harm to users or society**.

3.

Software Engineering Code of Ethics (ACM/IEEE)

4.

1. **8 Key Principles:**

    1. **Public:** Ensure software is safe and does not harm users.
    2. **Client & Employer:** Be honest about limitations and follow legal guidelines.
    3. **Product:** Ensure high quality, reliability, and maintainability.
    4. **Professional Judgment:** Make unbiased technical decisions.
    5. **Management:** Assign work fairly and ensure fair employment practices.

Downloaded by sana umeed (sanaumeed86@gmail.com)

6. **Profession:** Promote ethical software development.
7. **Colleagues:** Respect and assist fellow professionals.
8. **Self:** Continue learning and improving professional skills.

5.

Why is Ethics Important?

6.

1. Ensures **fairness, honesty, and responsibility** in software engineering.
2. Prevents **security breaches, unethical behavior, and software failures**.

Possible Exam Questions

1.

What are the key principles of the Software Engineering Code of Ethics?
Answer: Public safety, client honesty, product quality, professional judgment, ethical management, promoting the profession, respecting colleagues, and self-improvement.

2.
3.

Why is ethics important in software development?
Answer: Ethics prevent harm, ensure fairness, and promote professional integrity in software engineering.

4.

Week 10: DevOps

Key Topics:

1.

What is DevOps?

2.

1. Combines **Development (Dev)** and **Operations (Ops)** to automate and improve software deployment.

Downloaded by sana umeed (sanaumeed86@gmail.com)

3.

DevOps Lifecycle Stages:

4.

1. **Continuous Development:** Plan and code.
2. **Continuous Integration:** Merge code frequently.
3. **Continuous Testing:** Automate testing with tools like Selenium.
4. **Continuous Deployment:** Deploy updates automatically.
5. **Continuous Monitoring:** Track system health using tools like Splunk.

5.

DevOps Tools:

6.

1. **Version Control:** Git, GitHub
2. **Configuration Management:** Ansible, Puppet
3. **Monitoring:** Nagios, Prometheus
4. **CI/CD:** Jenkins

Possible Exam Questions:

1.

What are the key benefits of DevOps?
Answer: Faster software delivery, improved collaboration, reduced errors, and better system monitoring.

2.
3.

Explain Continuous Integration and Continuous Deployment.
Answer: CI ensures frequent merging of code, while CD automates deployment to production environments.

4.

Week 11: Software Testing

Key Topics:

Downloaded by sana umeed (sanaumeed86@gmail.com)

1.

Verification vs Validation

2.

1. **Verification:** Ensures software meets requirements before implementation.
2. **Validation:** Confirms the software functions as intended after development.

3.

Types of Testing:

4.

1. **White Box Testing:** Examines internal code structure.
2. **Black Box Testing:** Tests system functionality without looking at internal code.

5.

Cyclomatic Complexity

6.

1. Measures code complexity.
2. Formula: **CC = E - N + 2P** (Edges, Nodes, Program components).

Possible Exam Questions:

1.

What is the difference between verification and validation?
Answer: Verification checks if the system meets requirements, while validation ensures the system functions correctly.

2.
3.

What is the purpose of Cyclomatic Complexity?
Answer: It helps measure code complexity and determine the number of independent paths for testing.

4.

Downloaded by sana umeed (sanaumeed86@gmail.com)

Week 12: Version Control with Git

Key Topics:

1.

What is Version Control?

2.

1. Tracks changes in code and enables collaboration.
2. Examples: Git, SVN, CVS.

3.

Basic Git Commands:

4.

1. `git init` – Initializes a repository.
2. `git clone <URL>` – Creates a local copy of a repository.
3. `git add <file>` – Stages changes for commit.
4. `git commit -m "message"` – Saves a snapshot of changes.
5. `git push origin main` – Uploads changes to the remote repository.

5.

Branching in Git:

6.

1. `git branch <branch_name>` – Creates a new branch.
2. `git checkout <branch_name>` – Switches to another branch.

Possible Exam Questions:

1.

What is the difference between Git and GitHub?
Answer: Git is a version control system, while GitHub is a cloud-based hosting service for Git repositories.

2.
3.

Downloaded by sana umeed (sanaumeed86@gmail.com)

Explain the purpose of Git branching.
Answer: Branching allows developers to work on different features separately without affecting the main project.

Week 13: Kanban Practical

Key Concepts

1.

What is Kanban?

2.
1. A **visual project management tool** used to track work progress.
2. Helps teams **organize tasks and improve workflow efficiency**.
3.

Key Elements of Kanban in Asana

4.

1. **New Project Creation** – Setting up a project in Asana.
2. **Adding & Assigning Tasks** – Assigning tasks to team members.
3. **Setting Timelines** – Adding deadlines and milestones.
4. **Dashboard View** – Project managers monitor task completion.

Possible Exam Questions

1.

What is Kanban, and why is it used?
Answer: Kanban is a visual task management system used to track and improve workflow efficiency.

2.
3.

How does Asana help with Kanban task management?
Answer: Asana allows teams to create projects, assign tasks, and track progress using Kanban boards.

4.

Downloaded by sana umeed (sanaumeed86@gmail.com)

Week 13: DevOps Practical

Key Concepts

1.

What is DevOps?

2.

1. A methodology that combines **development and operations** to improve software delivery.
2. Uses **automation, collaboration, and continuous integration (CI/CD)**.

3.

Key DevOps Practices:

4.

1. **Version Control (Git, GitHub, GitLab)** – Tracks changes in the codebase.
2. **Configuration Management (Ansible, Chef, Puppet)** – Automates system setup.
3. **Containerization (Docker, Kubernetes)** – Runs applications in isolated environments.
4. **Monitoring (Prometheus, Grafana)** – Tracks system performance.

Possible Exam Questions

1.

What is the role of CI/CD in DevOps?
Answer: CI/CD automates code integration, testing, and deployment, ensuring fast and reliable software delivery.

2.
3.

What are some key DevOps tools?
Answer: Git, Jenkins, Ansible, Docker, Kubernetes, Prometheus, and Grafana.

4.

Downloaded by sana umeed (sanaumeed86@gmail.com)

Week 13: Git Practical

Key Concepts

1.

Basic Git Commands

2.

1. `git init` – Initializes a repository.
2. `git add <file>` – Adds files to the staging area.
3. `git commit -m "message"` – Saves a snapshot of changes.
4. `git push origin main` – Uploads changes to a remote repository.

3.

Git Workflow

4.

1. **New Project Creation:** Set up a new Git repository.
2. **Committing Changes:** Track changes using `git commit`.
3. **Pushing to Remote Repository:** Upload changes using `git push`.

Possible Exam Questions

1.

What is the purpose of `git commit`?
Answer: It records a snapshot of the staged changes in the repository.

2.
3.

What is the difference between `git add` and `git commit`?
Answer: `git add` stages changes, while `git commit` saves them permanently in the repository.

4.

Final Notes:

- These summaries cover **all key concepts** from your slides.
- The **exam questions** are designed to **help you prepare quickly**.

Downloaded by sana umeed (sanaumeed86@gmail.com)

- If you need more **detailed explanations, examples, or practice questions**, let me know!

Week 14: Unit Testing with JUnit

Key Topics:

1.

What is Unit Testing?

2.

1. Testing individual components of a software application.

3.

JUnit Testing Framework:

4.

1. JUnit is a **Java-based** unit testing tool.

5.

Test Case Design Techniques:

6.

1. **Black Box Testing:** Tests without knowing internal logic.
2. **White Box Testing:** Tests internal logic of the code.

7.

Good Test Case Characteristics:

8.

1. Catches errors effectively.
2. Is neither too simple nor too complex.
3. Covers all possible cases, including boundary conditions.

Possible Exam Questions:

Downloaded by sana umeed (sanaumeed86@gmail.com)

1.

What is the purpose of unit testing?
Answer: To validate the correctness of individual software components and catch errors early.

2.
3.

What is the difference between Black Box and White Box testing?
Answer: Black Box testing focuses on software functionality, while White Box testing examines internal logic.

4.

Week 15 & 16: Code Reviews & Static Analysis

Key Topics:

1.

Types of Reviews:

2.

1. **Peer Review:** Code is reviewed by another developer.
2. **Walkthroughs:** Developer guides the review team through the code.
3. **Inspections:** A formal process to find defects in code.

3.

Checklists for Code Review:

4.

1. Code quality and maintainability.
2. Adherence to coding standards.
3. Detection of security vulnerabilities.

Possible Exam Questions:

Downloaded by sana umeed (sanaumeed86@gmail.com)

What is the purpose of a code review?
Answer: To detect defects early, ensure code quality, and improve maintainability.

List different types of software reviews.
Answer: Peer reviews, walkthroughs, inspections, and audits.

Final Note:

This summary covers all major topics from your slides in an **easy-to-understand** manner. If you need more **explanations, examples, or additional questions**, let me know!

Downloaded by sana umeed (sanaumeed86@gmail.com)